

Multi-Agent CAD Code Generation

YANG LIU, Beijing Institute of Technology, China and Nanyang Technological University, Singapore

DAXUAN REN, Nanyang Technological University, Singapore

YIJIE DING, Nanyang Technological University, Singapore

JIANMIN ZHENG, Nanyang Technological University, Singapore

FANG DENG*, Beijing Institute of Technology, China

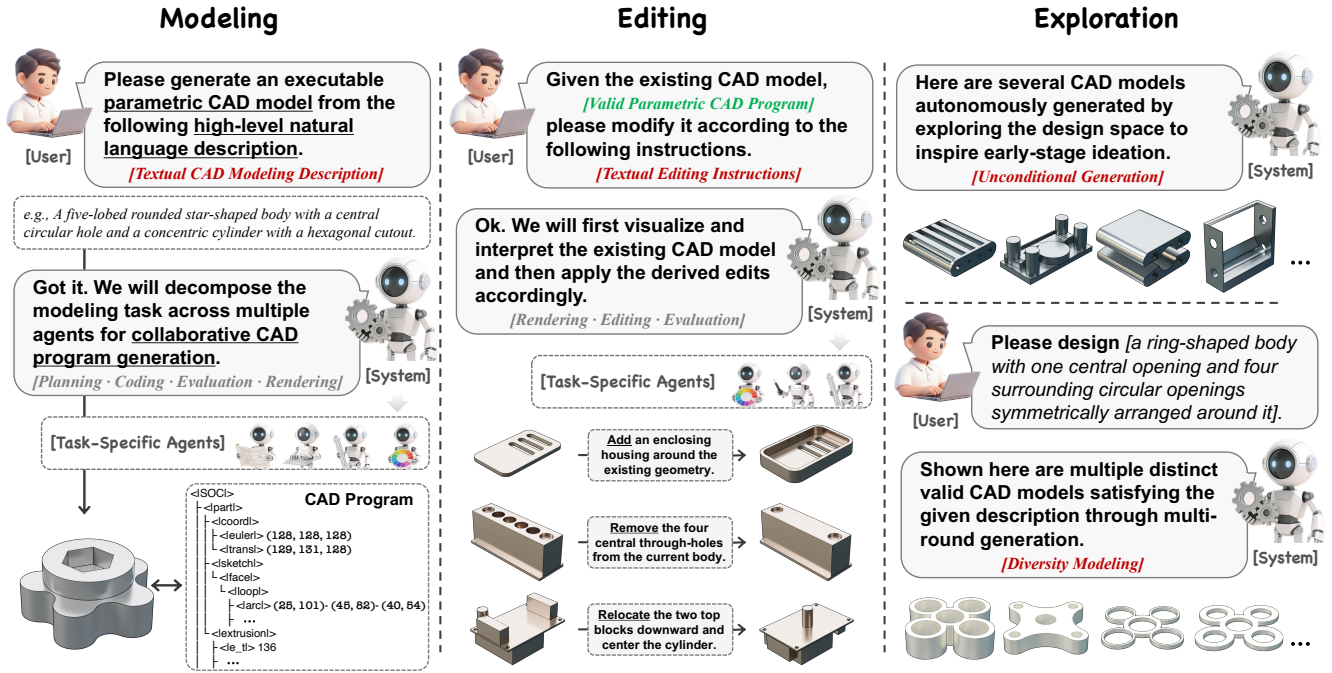


Fig. 1. **CAD-Factory: A language-driven CAD generation system.** The system translates natural language instructions into interpretable and editable parametric CAD models, supporting direct CAD program synthesis (modeling), and naturally extending to downstream applications including editing (language-guided modification) and exploration (unconditional or diverse design generation).

Advances in large language models (LLMs) have sparked interest in automating parametric CAD modeling through natural language. Existing LLM-based approaches often treat CAD modeling as flat text generation, overlooking the hierarchical structure and geometric constraints inherent in CAD programs. We present *CAD-Factory*, a Text-to-CAD generation system for language-driven CAD modeling that explicitly models the structural and

parametric semantics of CAD programs. Our core contribution is a new formulation of CAD generation as structured program synthesis, coupled with a learnable hierarchical CAD program representation that disentangles structural topology from parameter instantiation. Building on this formulation, *CAD-Factory* adopts a *manager-programmer-reviewer architecture*: a planning agent infers program hierarchy, a coding agent instantiates symbolic and numerical parameters, and an evaluation module enforces structural validity and geometric feasibility, which supports structure-aware reasoning, constraint-consistent generation, and interpretable program synthesis. An editor-viewer loop further enables part-aware code refinement through visual feedback, supporting iterative and controllable design workflows. We also contribute a process-oriented annotation pipeline and a Text-CAD dataset with parameter-free or partially specified descriptions that reflect real-world design expression. Extensive experiments demonstrate that *CAD-Factory* significantly improves structural correctness and geometric consistency across diverse CAD code generation and editing tasks, establishing a structured, interpretable, and robust framework for AI-assisted design.

*Corresponding author.

Authors' Contact Information: Yang Liu, Beijing Institute of Technology, Beijing, China and Nanyang Technological University, Singapore, Singapore, liuyang@bit.edu.cn; Daxuan Ren, Nanyang Technological University, Singapore, Singapore, daxuan001@e.ntu.edu.sg; Yijie Ding, Nanyang Technological University, Singapore, Singapore, yijie002@e.ntu.edu.sg; Jianmin Zheng, Nanyang Technological University, Singapore, Singapore, asjzhang@ntu.edu.sg; Fang Deng, Beijing Institute of Technology, Beijing, China, dengfang@bit.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGGRAPH Conference Papers '26, Los Angeles, CA, USA*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2554-8/2026/07
<https://doi.org/10.1145/3799902.3811067>

CCS Concepts: • **Computing methodologies** → **Symbolic and algebraic algorithms**; • **Theory of computation** → **Formal languages and automata theory**.

Additional Key Words and Phrases: Computer Aided Design, Multi-agent System, Parametric CAD Modeling, Large Language Model

ACM Reference Format:

Yang Liu, Daxuan Ren, Yijie Ding, Jianmin Zheng, and Fang Deng. 2026. Multi-Agent CAD Code Generation. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers (SIGGRAPH Conference Papers '26)*, July 19–23, 2026, Los Angeles, CA, USA. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3799902.3811067>

1 Introduction

Computer-aided design (CAD) transforms human design intent into executable and editable modeling programs that are interpreted to produce 3D geometry [Groover and Zimmers 1984]. During interactive design, users convey conceptual ideas via incremental modeling actions, whereas CAD systems instantiate these intents through ordered modeling operations with explicit parameter specifications. This intermediate process is realized as a sequence of parametric modeling instructions, in which atomic operations and Boolean operations are organized to progressively construct the solid shape [Shah and Mäntylä 1995]. From this perspective, the modeling process is inherently driven by structured operation planning and parameter instantiation.

Although modern CAD tools streamline the modeling workflow, high-quality designs still rely heavily on expert knowledge and manual effort. Prior work has explored recovering modeling sequences from external observations, such as point clouds [Dupont et al. 2024; Khan et al. 2024a], images [Chen et al. 2025c], or hand-drawn sketches [Li et al. 2025c], with promising progress in reverse engineering. These approaches typically assume sufficient geometric information and focus on reproducing existing designs. Recently, language-driven CAD generation has attracted increasing interest as a potential approach to express design intent [Khan et al. 2024b; Li et al. 2025a], enabling design specification in terms of high-level semantics and functional requirements. However, translating textual descriptions into executable and structurally coherent CAD programs remains challenging, as it requires grounding abstract semantics, resolving linguistic ambiguity, and enforcing consistency with valid modeling operations.

Large language models (LLMs) are naturally suited to this task, given their strengths in semantic reasoning, sequential planning, and code generation, and can be instantiated as specialized agents for different roles. Leveraging their vast knowledge, prior studies have demonstrated the feasibility of generating CAD programs under zero-shot or few-shot settings [Ocker et al. 2025; Wang et al. 2025a], and have further incorporated multimodal inputs to enhance geometric and semantic understanding [Xu et al. 2025; Yavartanoo et al. 2024], while designing task-specific models for particular design contexts [Li et al. 2024; Zhang et al. 2025]. Nevertheless, current paradigms still suffer from fundamental limitations: (1) *most methods operate as one-pass processes without considering the inherent hierarchical structure of parametric CAD programs*, and (2) *they commonly assume that textual descriptions provide sufficient semantic or geometric cues (e.g., detailed primitives or parameters)*, deviating from the abstract and under-constrained nature of language-based design. These mismatches often lead to incoherent structure and

inconsistent parameterization, constraining their applicability in real-world CAD workflows.

In this paper, we introduce CAD-Factory, a language-driven CAD generation system that addresses the above challenges from two complementary perspectives. First, we revisit CAD design workflows and decompose the generation process into structured stages via dedicated agents with distinct functional roles for structure planning, code generation, and program editing, enabling flexible and controllable generation with interactive feedback across multiple tasks (Figure 1) within a unified framework. Second, we exploit the hierarchical nature of parametric CAD programs and adopt an abstract syntax tree (AST) representation as a domain-specific language (DSL)-style abstraction, enabling the generation process to first establish a coherent “program skeleton” and then progressively instantiate detailed geometric parameters from abstract textual descriptions. Rather than serving as a new CAD language, the AST is used as an intermediate representation for staged CAD program synthesis, explicitly organizing part hierarchy, operation ordering, and dependency structure before parameter completion. In addition, we incorporate modeling steps and program evolution into the construction of Text–CAD pairs, aligning LLM learning with real design processes.

The primary contributions of the paper are threefold.

- We frame language-driven CAD generation as a structured program synthesis paradigm and present CAD-Factory as a unified framework supporting modeling, editing, and design exploration from abstract textual descriptions.
- We provide a learnable hierarchical CAD program representation based on an abstract syntax tree, which serves as an intermediate representation for staged CAD generation by explicitly organizing structural planning before parameter instantiation.
- We develop a process-oriented annotation pipeline and construct a multimodal Text–CAD dataset to better align CAD generation with the procedural nature of real-world design workflows.

Extensive experiments demonstrate CAD-Factory’s superior performance in under-constrained text-to-CAD generation, while highlighting its robustness and flexibility across diverse downstream tasks. The project page is available at cad-factory.github.io.

2 Related Work

2.1 Parametric CAD Modeling.

Parametric modeling defines shapes via structured representations that jointly encode geometry and construction semantics. While substantial progress has been made in boundary representation modeling [Fu et al. 2025; Guo et al. 2022; Lambourne et al. 2021; Li et al. 2025d; Xu et al. 2024] and constructive solid geometry generation [Ren et al. 2021, 2022], recent studies have increasingly shifted toward learning procedural construction instructions. This line of work casts CAD modeling as a sequence prediction problem, in which modeling commands (e.g., sketching and extrusion) are progressively inferred to construct the target shape. Recent advances in deep learning have further enabled reliable reconstruction of parametric CAD instructions from scanned or raster data (e.g., sketches, images, or point clouds) [Chen et al. 2025b; Li et al. 2022; Yang et al. 2025a]. In contrast to these geometry-oriented formulations, our

work focuses on language-driven CAD generation, where parametric instructions are derived from abstract textual descriptions that more closely reflect how design intent is expressed in real-world design workflows.

2.2 Large Multimodal Models.

LLMs have emerged as powerful reasoners, demonstrating strong capabilities in understanding, planning, and generation across diverse domains. A prominent application of LLMs lies in code synthesis, where natural language is translated into executable programs by modeling complex syntax and semantics [Nijkamp et al. 2023; Roziere et al. 2024]. Related directions further explore multi-agent and task decomposition frameworks, which improve generation stability by decomposing complex tasks into subproblems [Hong et al. 2024; Huang et al. 2023; Wu et al. 2024], and have been applied across a range of generative tasks [Chen et al. 2025a; Li et al. 2025b]. More recently, multimodal large language models (MLLMs) extend these strengths by integrating visual information, enabling joint language–vision reasoning for tasks such as visual question answering, grounded instruction following, and layout understanding [Li et al. 2023; Radford et al. 2021]. Building on these advances, our work follows this trend of structured and decomposed generation by developing domain-adapted LLMs for parametric CAD program generation and leveraging MLLMs for Text–CAD pair annotation.

2.3 LLMs and MLLMs for CAD Generation.

Recent breakthroughs in large multimodal models have stimulated growing interest in their application to CAD domains. LLMs are employed to translate textual descriptions into structured modeling commands, enabling a more intuitive and accessible language-driven design workflow [Du et al. 2024]. Through large-scale pretraining, LLMs internalize rich shape semantics and symbolic construction priors, substantially lowering the barrier to learning and producing parametric CAD instructions [Guan et al. 2025; Rukhovich et al. 2025]. Concurrently, recent efforts have explored diverse directions for improving CAD generation, ranging from structured and DSL-based representations to feedback-driven and multi-agent generation paradigms [Elrefaie et al. 2025; Fan et al. 2025; Jones et al. 2025; Wang et al. 2026]. Building on this foundation, MLLMs further integrate visual context into their reasoning process, enabling multimodal-conditioned CAD modeling with improved controllability and adaptability [Mallis et al. 2025; Wang et al. 2025c]. Unlike prior approaches that primarily rely on single-pass paradigms, i.e., directly mapping language inputs to complete CAD programs, our work introduces a staged generation system that separates structure planning from parameter completion, enabling more coherent and reliable CAD generation.

3 Natural Language CAD Synthesis

The problem addressed in this paper is to develop a language-driven CAD generation system that converts design descriptions $\mathcal{T}$ —specifying target geometry and structure—into executable CAD programs C , i.e., $\mathcal{T} \rightarrow C$. Each program defines a parametric CAD model $\mathcal{M}$ as an ordered sequence of modeling instructions, where parts are constructed via sketch profiles and extrusion operations,

Table 1. Token categories and definitions in CAD AST representation.

Category	Definition
Special Tokens	< SOC >, < EOC >, < part >
Coordinate Tokens	< coord >, < euler >, < trans >
Sketch Tokens	< sketch >, < face >, < loop >, < line >, < arc >, < circle >
Extrusion Tokens	< extrusion >, < e_t >, < e_o >, < size >, < operation >, < New >, < Join >, < Cut >, < Inter >

and assembled through Boolean operations to form the final geometry. Formally, this task is defined as a conditional generation process: $C \sim p_\theta(C | \mathcal{T})$. However, a CAD program is not a simple linear sequence like natural language, but implicitly encodes a hierarchical structure that specifies part decomposition, operation ordering, and inter-part dependencies. Collapsing this hierarchy into an end-to-end text-to-code mapping obscures structural reasoning and leads to brittle generation when textual inputs lack sufficient constraints.

Given this insight, we explicitly disentangle the inherent structure of CAD models into an intermediate representation $\mathcal{A}$ that encodes overall modeling organization independent of geometric parameters. This representation serves as a structural scaffold, capturing part organization, operation ordering, and inter-part dependencies while abstracting away low-level geometric parameters. The generation process is thus reformulated as $\mathcal{T} \rightarrow \mathcal{A} \rightarrow C$, where $\mathcal{A}$ is first inferred from $\mathcal{T}$, and geometric parameters are then specified to obtain C . From a probabilistic perspective, this process can be expressed as:

$$p_\theta(C | \mathcal{T}) = \sum_{\mathcal{A}} p_\theta(C | \mathcal{A}, \mathcal{T}) p_\theta(\mathcal{A} | \mathcal{T}). \quad (1)$$

This factorization explicitly separates high-level structural reasoning from low-level parameter instantiation, yielding a systematic formulation for CAD generation.

3.1 Overview

To realize the above goal, we introduce **CAD-Factory**, a framework for language-driven CAD generation that decomposes the problem into structured stages aligned with practical CAD workflows (Figure 2). Instead of a direct text-to-CAD mapping, the generation process is decomposed into functional components for structure planning, parameter coding, executability validation, visualization, and interactive editing. At its core, an AST-based intermediate representation explicitly encodes part hierarchy, operation ordering, and dependencies, enabling a structure-aware progression from text to executable CAD code. We further design a process-oriented annotation pipeline and build a Text–CAD dataset for system optimization. The core techniques are elaborated in the rest of this section, with additional details provided in Appendix C.

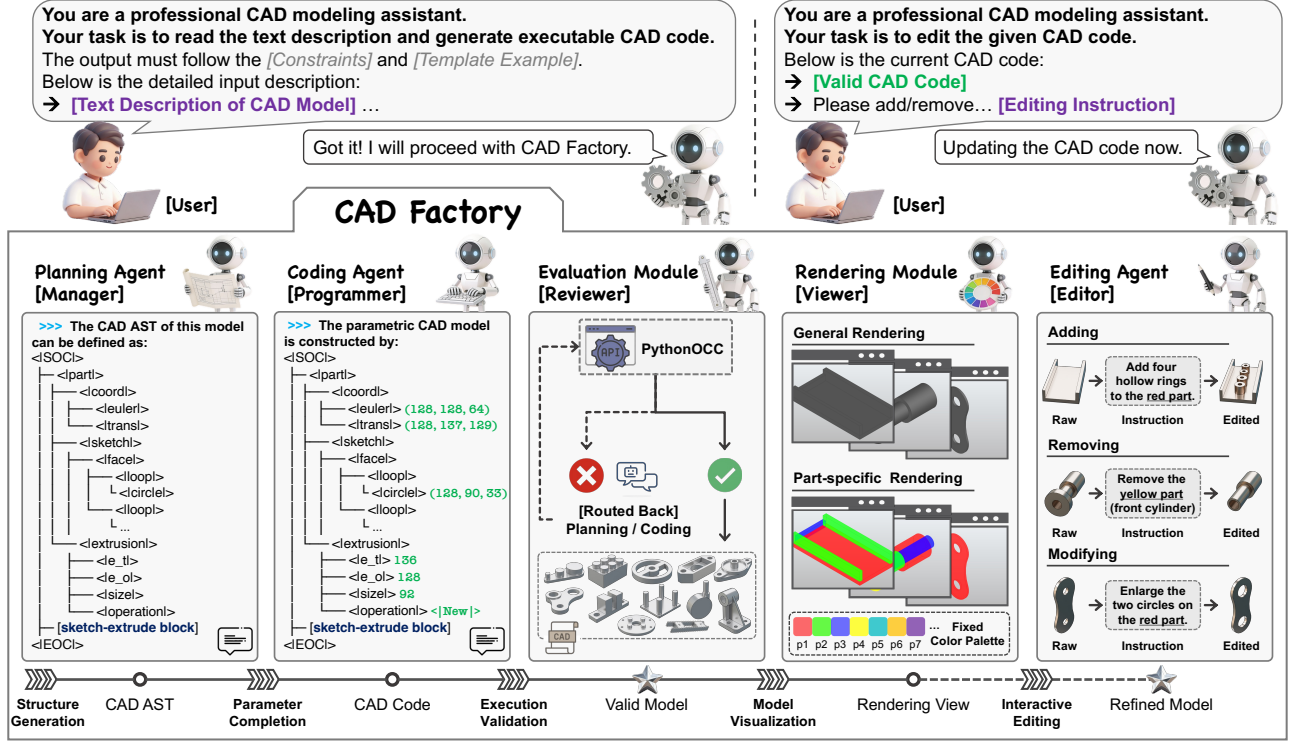


Fig. 2. CAD-Factory is a language-driven CAD generation system, comprising modules for planning, coding, evaluation, rendering, and editing. It decomposes generation into structure planning and parameter completion, followed by execution validation to ensure valid CAD models. Interactive editing is enabled through targeted instructions, with general and part-specific renderings providing intuitive visual guidance for precise modifications. The star symbol denotes stages that produce model outputs.

3.2 AST Representation

Inspired by structured representations in program compilation, we introduce a learnable, domain-specific AST representation $\mathcal{A}$ tailored for parametric CAD modeling, which explicitly encodes the hierarchy and semantics of geometric primitives and modeling operations. Unlike flat CAD token sequences, which entangle part organization, operation order, and detailed parameters in a single autoregressive stream, the AST representation isolates the program-level scaffold before numerical instantiation. This abstraction is particularly useful in language-driven CAD generation, where textual descriptions often specify only high-level design intent and leave construction details underspecified. As summarized in Table 1, we define 21 composable AST nodes grouped into four functional categories: (1) special tokens that mark program boundaries and structural units; (2) sketch nodes that represent 2D profile primitives; (3) coordinate nodes that define local coordinate systems for sketch planes; and (4) extrusion nodes that control extrusion direction (toward $\langle e_t \rangle$ or opposite $\langle e_o \rangle$ the surface normal), scaling, and Boolean operations determining interactions with existing geometry. Concretely, the AST preserves structural units and their dependencies—such as part boundaries, local coordinate frames, sketch composition, and extrusion operators—while abstracting away low-level geometric values that are deferred to the coding stage. Under

this representation, each CAD model follows a consistent “coordinate $\rightarrow$ sketch $\rightarrow$ extrusion” pattern, yielding a structurally faithful and interpretable backbone for subsequent reasoning and parameterization.

3.3 CAD-Factory

Textual design descriptions typically convey only high-level design intent while leaving low-level construction details implicit, forcing LLMs to jointly reason about structural layout and geometric instantiation in a single pass, which often leads to inconsistent or infeasible outputs. Motivated by this observation, we decouple CAD generation into structure planning and parameter completion, allowing each stage to be explicitly handled and progressively refined within a staged generation framework. Several components are further introduced for validation, visualization, and interactive editing, forming a unified system that supports diverse CAD applications.

3.3.1 Planning Agent. To explicitly construct the global structure of a CAD model, we introduce a dedicated planning agent $\mathcal{G}_{\text{plan}}$ that acts as an architectural manager for *Text-to-AST generation* (Figure 2, left column). Given an input description $\mathcal{T}$, the agent infers an intermediate AST $\mathcal{A}$ that captures part organization, operation ordering, and dependency relations, formulated as $\mathcal{A} = \mathcal{G}_{\text{plan}}(f_{\text{enc}}(\mathcal{T}))$, where $f_{\text{enc}}(\cdot)$ denotes the encoding process realized by a learnable

tokenizer and text encoder. The resulting AST provides a structured and interpretable blueprint that constrains the overall program layout and guides subsequent parameterization.

3.3.2 Coding Agent. Guided by the above structural skeleton, the coding agent $\mathcal{G}_{\text{code}}$ serves as a programmer responsible for *AST-to-Code completion* (Figure 2, second column). It instantiates all required numerical and symbolic attributes under the structural constraints imposed by $\mathcal{A}$, thereby ensuring semantic consistency and geometric validity. In practice, the agent takes the inferred AST together with the original textual description as inputs and produces the final parametric CAD code as $C = \mathcal{G}_{\text{code}}(\mathcal{A}, f_{\text{enc}}(\mathcal{T}))$. The generated code is directly executable for downstream validation and rendering. Notably, both $\mathcal{G}_{\text{plan}}$ and $\mathcal{G}_{\text{code}}$ can leverage the same LLM backbone but are instantiated independently for complementary system roles.

3.3.3 Evaluation Module. To ensure the reliability of generated CAD programs, we employ an evaluation module that performs both structural and parametric verification. It checks the inferred AST for token ordering, hierarchical consistency, and completeness, and validates parameter feasibility through the PythonOCC CAD engine (Figure 2, third column). Detected errors are fed back to the planning or coding agent for refinement, with up to two feedback rounds allowed in our experiments.

3.3.4 Rendering Module. To provide clear and consistent visual feedback, a rendering module $\mathcal{G}_{\text{render}}$ is provided to visualize the generated CAD models. Since traceability from code blocks to solid parts is non-trivial, we augment general rendering with part-specific visualization using a fixed color-mapping scheme. This design ensures that parts with the same index are rendered in a consistent, predefined color across different models (Figure 2, fourth column), facilitating part identification and intuitive downstream editing.

3.3.5 Editing Agent. To support interactive CAD refinement, we introduce an editing agent $\mathcal{G}_{\text{edit}}$ that enables instruction-guided operations such as *adding*, *removing*, and *modifying*, with *querying* as a prerequisite (Figure 2, right column). Given an original program C and an editing instruction $\mathcal{P}_{\text{edit}}$, the agent produces an updated program $C' = \mathcal{G}_{\text{edit}}(C; \mathcal{P}_{\text{edit}})$, where $\mathcal{P}_{\text{edit}} = (\mathcal{E}_{\text{edit}}, \pi_{\text{color}})$ consists of the editing intent $\mathcal{E}_{\text{edit}}$ and a color-to-part index mapping π_{color} . The target editing scope is identified from part-specific renderings (e.g., a red solid corresponds to part 1 in the CAD program), with the color-to-part correspondence established via one-shot prompting. In addition, we construct an editing dataset by applying controlled modifications to existing Text-CAD data and fine-tune the agent using the same strategy as the planning and coding agents. See Appendix B.3 for more editing implementation details.

3.4 Optimization

3.4.1 CAD AST Learning. Off-the-shelf LLMs, while effective for general language tasks, typically lack geometric structural priors and have not been exposed to CAD-specific AST representations. To bridge this gap, we curate a specialized CAD corpus consisting of paired textual descriptions and corresponding ASTs to facilitate structure-aware learning. Rather than treating these tokens

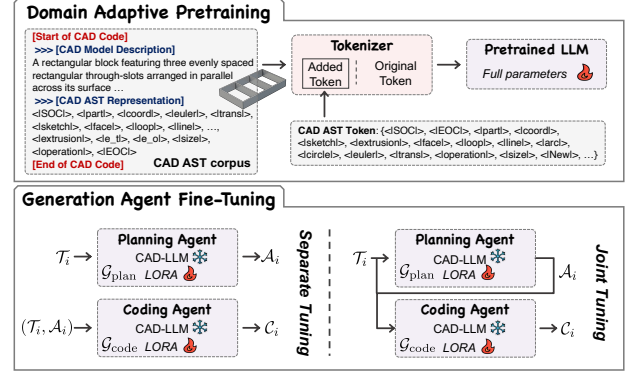


Fig. 3. The training pipeline consists of domain-adaptive pretraining on a structured CAD AST corpus to inject geometric priors, followed by agent-specific fine-tuning for structure planning and parameter coding. A joint tuning stage further aligns both agents for coherent CAD code generation.

as ordinary text, we integrate them into the tokenizer to preserve symbol-level uniqueness. To provide each token s with a semantically informative initialization, its embedding is derived from a small set of descriptive keywords $\mathcal{K}^{(s)} = \{k_1^{(s)}, k_2^{(s)}, \dots, k_{m_s}^{(s)}\}$:

$$\mathbf{E}(s) = \frac{1}{\sum_{i=1}^{m_s} |k_i^{(s)}|} \sum_{i=1}^{m_s} \sum_{j=1}^{|k_i^{(s)}|} \mathbf{E}(w_j^{(i)}) \quad (2)$$

where $w_j^{(i)}$ denotes the j -th subword token of keyword $k_i^{(s)}$, and $\mathbf{E}(\cdot)$ is the embedding function. This strategy aligns AST token representations with their functional roles in CAD modeling, promoting consistent structural parsing (see Appendix C.1).

3.4.2 Domain-adaptive Pretraining. As illustrated in Figure 3 (top), we perform full-parameter domain-adaptive pretraining on the extended vocabulary using the CAD AST corpus. Since different CAD models may share similar structural patterns while differing in parameter instantiation, we group samples by block-level AST similarity and adopt a curriculum learning strategy. Specifically, we treat $\langle |\text{loop}| \rangle$ as the minimal unit and progressively aggregate samples by matching token types and counts, extending hierarchically through the $\langle |\text{face}| \rangle$, $\langle |\text{sketch}| \rangle$, and $\langle |\text{part}| \rangle$ levels. Pretraining is conducted using an autoregressive objective, where the model predicts each token conditioned on preceding context:

$$\mathcal{L}_{\text{AST}} = \sum_{t=1}^n \log P(x_t | x_{0:t-1}) \quad (3)$$

with x_n denoting the end-of-CAD token $\langle |\text{EOC}| \rangle$. This stage equips the model with geometry-aware structural priors, facilitating stable hierarchical reasoning and faithful CAD program generation.

3.4.3 Agentic Fine-Tuning. Given the domain-adapted LLM, we first fine-tune the planning and coding agents separately on their respective subtasks (Figure 3, bottom-left). The planning agent learns to infer structural patterns $\mathcal{A}$ from input text $\mathcal{T}$, while the coding agent completes CAD code C by instantiating numerical and symbolic parameters conditioned on both $\mathcal{T}$ and $\mathcal{A}$. We adopt LoRA [Hu et al. 2022] for parameter-efficient adaptation of both agents with

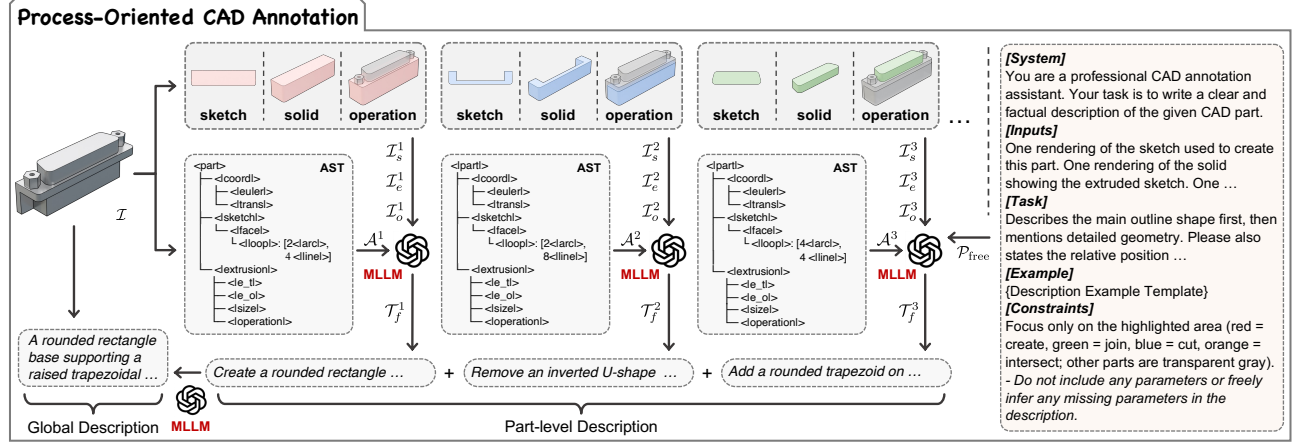


Fig. 4. **Process-Oriented Annotation Pipeline.** Each CAD part is first annotated based on its sketch $\mathcal{I}_s^i$, extrusion $\mathcal{I}_e^i$, and operation $\mathcal{I}_o^i$ renderings, together with the corresponding AST $\mathcal{A}^i$, to produce parameter-free part-level descriptions $\mathcal{T}_f^i$ via an MLLM. These descriptions are then sequentially aggregated and further processed by the MLLM, conditioned on the complete rendering $\mathcal{I}$, to produce a process-aligned global description.

objectives defined as:

$$\mathcal{L}_{\text{plan}} = \sum_{i=1}^N \log P(\mathcal{A}_i | \mathcal{T}_i), \quad \mathcal{L}_{\text{code}} = \sum_{i=1}^N \log P(C_i | \mathcal{T}_i, \mathcal{A}_i). \quad (4)$$

This agent-specific tuning allows each agent to specialize in its distinct role. We then perform joint fine-tuning (Figure 3, bottom-right) to align the collaborative behavior of the planning and coding agents. The joint objective is defined as:

$$\mathcal{L}_{\text{joint}} = \sum_{i=1}^N [\log P(\mathcal{A}_i | \mathcal{T}_i) + \log P(C_i | \mathcal{T}_i, \mathcal{G}_{\text{plan}}(\mathcal{T}_i))]. \quad (5)$$

3.5 Process-Oriented Text-CAD Dataset

Despite growing interest in text-guided CAD modeling, most existing Text-CAD datasets rely on overly detailed descriptions that explicitly specify low-level geometric parameters, diverging from how design intent is typically expressed in practice. Real-world design communication is often abstract, partial, and under-constrained. Although some efforts aim to provide higher-level textual descriptions for CAD models, they often suffer from semantic inconsistencies caused by imperfect annotation pipelines or (M)LLMs hallucinations. To bridge this gap, we introduce a process-oriented annotation pipeline and construct a new Text-CAD dataset that pairs CAD programs with concise, parameter-free descriptions, better aligning language supervision with practical CAD design workflows.

3.5.1 Data Collection. We adopt the DeepCAD dataset [Wu et al. 2021] as the base source, which contains $\sim 180\text{K}$ parametric CAD models represented in JSON-based programs. Duplicate entries and trivial geometries (e.g., simple cubes or cylinders) are filtered out for stable model training, resulting in $\sim 110\text{K}$ samples.

3.5.2 Dataset Contents. Although our work primarily focuses on Text-to-CAD code generation, we also provide multimodal data to support flexible annotation and broaden potential applications. Specifically, each CAD model is accompanied by a set of visual representations, including a complete assembly image $\mathcal{I}$ and part-level

views corresponding to different modeling stages: sketch profiles $\mathcal{I}_s^i$, extruded geometries $\mathcal{I}_e^i$, and assembly operation effects $\mathcal{I}_o^i$, where i denotes the part index. Beyond the original CAD program C , each model is paired with an AST $\mathcal{A}$, which preserves the hierarchical structure of the parametric CAD model without low-level parameter details.

3.5.3 Annotation Pipeline. For text-to-CAD generation, textual descriptions should convey global modeling intent while remaining abstract, sequential, and consistent with natural design expression. Rather than applying one-step captioning to the final CAD geometry, we propose a process-oriented annotation pipeline tailored to step-wise parametric CAD modeling, where descriptions are generated conditioned on local geometric renderings and modeling structures (Figure 4). This design better reflects the structured and sequential nature of CAD construction and helps reduce hallucinations caused by free-form description of the final shape.

To better align with the under-constrained nature of real-world design expression, we adopt a parameter-free annotation strategy, i.e., focusing on capturing high-level modeling intent without exposing explicit geometric parameters, and treat each CAD model as an ordered sequence of construction steps. For each step i , we leverage the corresponding part-level renderings together with its decomposed AST node $\mathcal{A}^i$, and prompt an MLLM to generate a concise, semantically coherent, parameter-agnostic description aligned with that modeling stage:

$$\mathcal{T}_f^i = \text{MLLM}(\mathcal{I}_o^i, \mathcal{I}_e^i, \mathcal{I}_s^i, \mathcal{A}^i; \mathcal{P}_{\text{free}}) \quad \text{for } i = 1, \dots, n, \quad (6)$$

where $\mathcal{P}_{\text{free}}$ enforces abstraction by excluding explicit geometric parameters. The description $\mathcal{T}_f$ is constructed by concatenating all part-level texts in operation order, forming a narrative that reflects the procedural nature of CAD modeling. Compared with one-step captioning over the final geometry, this staged construction preserves intermediate modeling semantics and reduces the risk of introducing unsupported structural descriptions. To provide holistic design intent, we further prompt the MLLM with the aggregated

part-level descriptions together with a complete assembly image I to generate a concise global description, which is prepended to the sequence to constitute the final textual description.

To improve annotation quality, we apply automated consistency checks by aligning the number of modeling-step descriptions with the underlying CAD program structure. Challenging cases with obvious inconsistencies or missing semantics are manually inspected and revised by experienced CAD users. In addition, we also construct a parameter-partial annotation variant by introducing a limited set of intuitive parameter cues (e.g., extrusion distance) while excluding low-level sketch specifications, as detailed in Appendix B.2. All annotations are generated using GPT-4o, with implementation details in Appendix B.

4 Experiments

4.1 Experimental Setup

4.1.1 Dataset. We evaluate our method on the proposed Text-CAD dataset, which contains $\sim 110K$ samples, each paired with a textual description, an AST representation, and parametric CAD code. For text-to-CAD generation, we randomly sample 50K entries for tuning. For instruction-based editing, we sample 5K entries for each operation—adding, removing, and modifying—yielding a 15K-sample subset. All experiments in the main text are conducted under the parameter-free setting, while evaluations under the parameter-partial setting are reported separately in Appendix D.3–D.4.

4.1.2 Implementation Details. We adopt LLaMA-3.2-3B [Meta 2024] as the base LLM for the planning, coding, and editing agents. During CAD domain-adaptive pretraining, we perform full-parameter tuning for 3 epochs with a context length of 2,048 tokens. For code generation and editing, we apply LoRA layers [Hu et al. 2022] to each agent with rank 32 and $\alpha = 64$, using an extended context window of 4,096 tokens for 5 epochs. All stages are optimized using AdamW optimizer [Loshchilov and Hutter 2019] with a learning rate of $2e-5$, a linear scheduler, and 1,000 warm-up steps. FlashAttention [Dao 2024] is employed to improve training efficiency.

4.1.3 Baselines. We compare our method against a comprehensive set of baselines spanning diverse paradigms for text-to-CAD generation: *General-purpose LLMs* include GPT-4 [Achiam et al. 2024], Gemini-1.5 [Team et al. 2024], DeepSeek-V3 [Liu et al. 2025], and Qwen-3-14B [Yang et al. 2025b]; *Code-oriented LLMs*, represented by Claude-4, are designed for structured program synthesis; *Multi-modal methods*, including 3D-PreMise [Yuan et al. 2024] and CADVerify [Alrashedy et al. 2025], incorporate visual information to assist CAD generation; and *Task-specific methods*, such as Text2CAD [Khan et al. 2024b] and CADFusion [Wang et al. 2025b], are designed for text-guided CAD modeling with specialized architectures and training objectives. For unconditional generation, we compare with representative methods, including DeepCAD [Wu et al. 2021], SketchGen [Para et al. 2021], and HNC-CAD [Xu et al. 2023], which are widely used benchmarks for evaluating structural validity, diversity, and novelty in parametric CAD generation. Implementation details for all baselines are provided in Appendix C.2–C.3.

4.1.4 Evaluation Metrics. To comprehensively evaluate our framework across various CAD generation tasks, we adopt a set of metrics

Table 2. Text-to-CAD generation in *parameter-free* setting. ACC_{ast} and F1 are reported in percentage (%). MCD, MMD, and JSD are scaled by 10^2 .

Method	$ACC_{ast} \uparrow$	F1 $\uparrow$	MCD $\downarrow$	MMD $\downarrow$	JSD $\downarrow$
<i>General-Purpose LLMs</i>					
GPT-4	47.40	31.69	25.66	3.44	16.70
Gemini-1.5	47.15	36.19	37.75	4.93	17.94
DeepSeek-V3	39.89	35.94	21.85	4.18	11.57
Qwen-3-14B	32.19	36.57	20.14	4.69	13.96
<i>Coding-Oriented LLM</i>					
Claude-4	54.36	33.79	19.03	3.51	11.03
<i>Multi-modal Methods</i>					
3D-PreMise	45.31	36.92	25.41	3.65	8.75
CADVerify	44.49	37.08	21.95	3.22	9.77
<i>Task-Specific Methods</i>					
Text2CAD	75.18	51.96	22.99	2.95	9.56
CADFusion	56.24	48.72	12.35	3.19	8.30
CAD-Factory	82.81	76.06	2.90	1.22	1.18

that assess both generated programs and the resulting geometry. For text-to-CAD generation, following [Khan et al. 2024b], we report F1-score, Median Chamfer Distance (MCD), Minimum Matching Distance (MMD), and Jensen–Shannon Divergence (JSD) to measure parameter accuracy and geometric consistency. We further introduce AST accuracy (ACC_{ast}) as a structural metric to directly evaluate the correctness of the generated abstract syntax tree. For unconditional generation, we follow prior works [Para et al. 2021; Wu et al. 2021] and report Coverage (COV), MMD, JSD, Invalid Rate (IR), and Novelty Score (Novel) to jointly assess diversity, realism, validity, and novelty. Detailed metric definitions and evaluation protocols are provided in Appendix C.4.

4.2 Experimental Results

4.2.1 Text-to-CAD Generation. As shown in Table 2, CAD-Factory consistently outperforms all baselines across all metrics. The substantial improvement in ACC_{ast} highlights a stronger ability to infer hierarchical CAD structures, which directly leads to more accurate parameter prediction, as reflected by higher F1 scores. Lower MCD, MMD, and JSD further indicate superior geometric fidelity and closer alignment with the real data distribution. Qualitative comparisons in Figure 5 show that our method produces structurally coherent CAD models with finer geometric details that better match the intended design semantics. Additional results under the parameter-partial setting are reported in Appendix D.3.

4.2.2 Cross-dataset Validation. To evaluate generalization to out-of-distribution data, we conduct cross-dataset validation on the Fusion 360 Gallery dataset [Willis et al. 2021]. Following the same procedure for obtaining textual descriptions, we directly apply the trained models without additional fine-tuning. Quantitative results are reported in Table 3, with qualitative comparisons shown in Figure 6. As shown, our method maintains strong performance across all criteria, demonstrating robust generalization to unseen CAD distributions. Notably, (M)LLM-based methods exhibit more stable

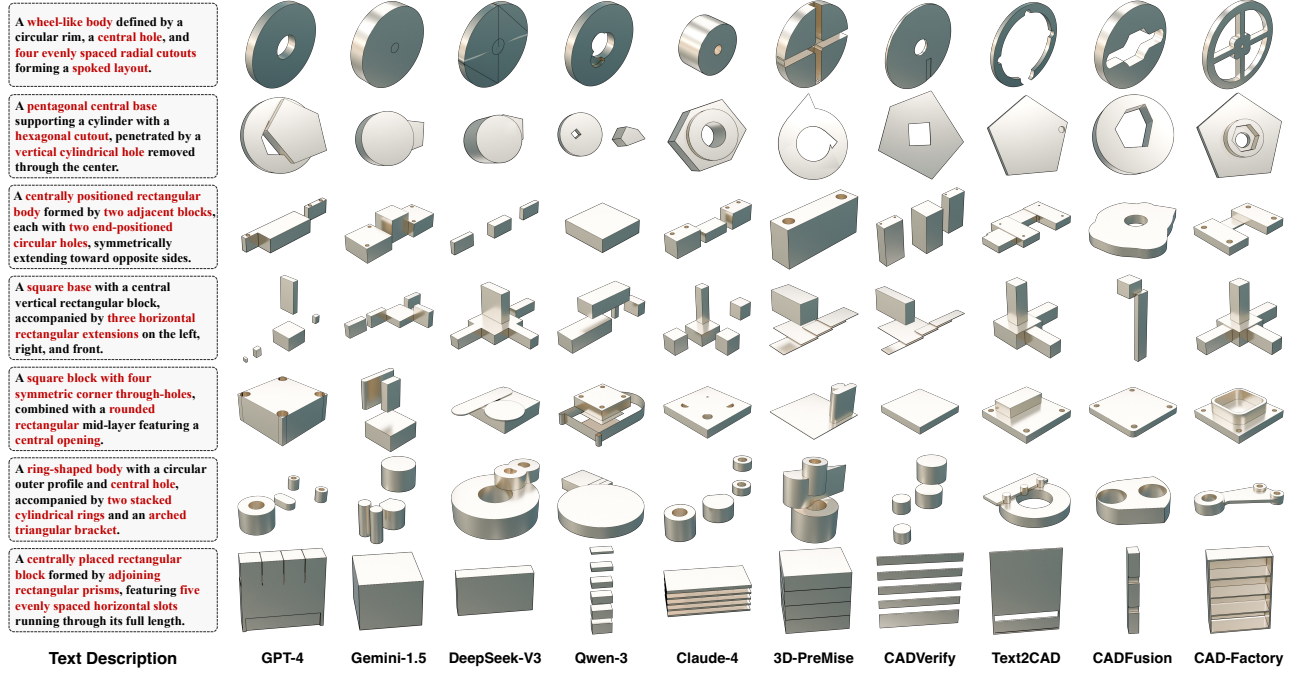


Fig. 5. Qualitative results of text-to-CAD generation in the *parameter-free* setting. Our method produces structurally coherent assemblies and well-formed geometries across diverse inputs, whereas baseline methods often generate models that roughly match the textual descriptions but exhibit fragmented structures or less plausible geometric layouts.

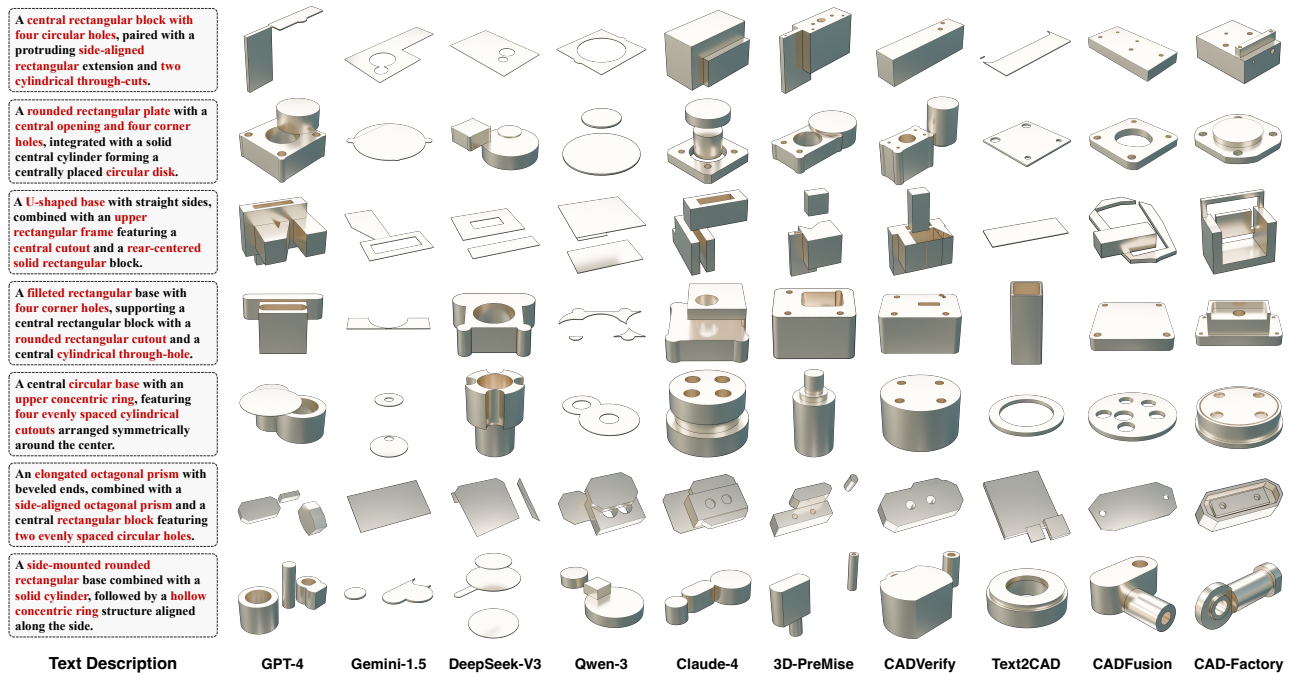


Fig. 6. Qualitative results of text-to-CAD generation in the *parameter-free* setting under cross-dataset validation on the Fusion 360 Gallery. Our approach generates CAD models that align well with the textual descriptions while preserving fine-grained geometric details, even for out-of-distribution data.

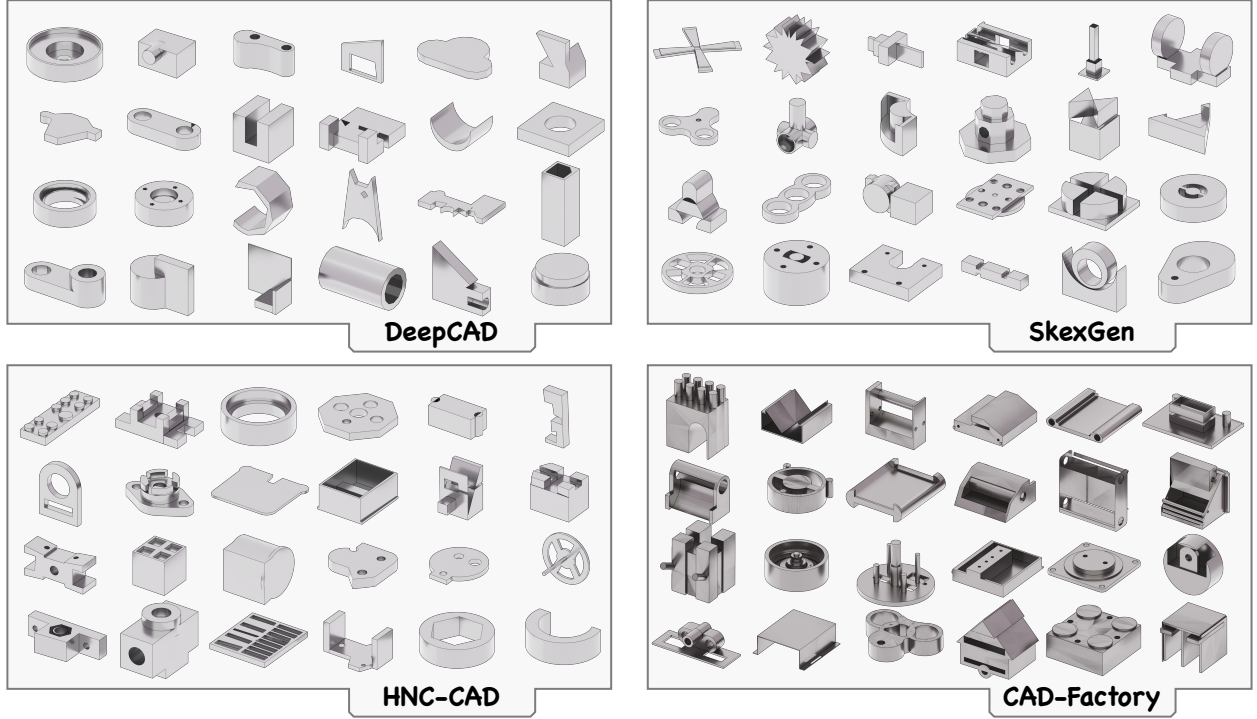


Fig. 7. Qualitative results of unconditional CAD generation. Models generated by our method exhibit increased structural complexity and design diversity.

Table 3. Text-to-CAD generation in *parameter-free* setting under cross-dataset validation on the Fusion 360 Gallery.

Method	ACC _{ast} ↑	F1 ↑	MCD ↓	MMD ↓	JSD ↓
<i>General-Purpose LLMs</i>					
GPT-4	45.85	36.48	22.75	4.34	16.33
Gemini-1.5	39.33	36.40	31.17	5.17	20.06
DeepSeek-V3	41.06	32.34	26.78	4.92	17.63
Qwen-3-14B	31.50	31.74	23.35	4.30	19.78
<i>Coding-Oriented LLM</i>					
Claude-4	46.44	36.16	19.64	3.74	13.90
<i>Multi-modal Methods</i>					
3D-PreMise	46.77	41.84	20.01	3.62	9.04
CADVerify	47.11	39.51	16.93	3.90	9.15
<i>Task-Specific Methods</i>					
Text2CAD	58.13	36.30	24.76	4.91	19.26
CADFusion	45.35	38.17	22.59	5.51	13.45
CAD-Factory	70.85	62.26	8.92	2.69	2.66

cross-dataset performance than task-specific approaches, suggesting an inherent generalization advantage from large-scale pretraining. More results are provided in Appendix D.4.

4.2.3 Unconditional Generation. While our work focuses on text-to-CAD modeling, it can be readily adapted to unconditional generation by using the prefix prompt “*Start of CAD Code*” from the CAD corpus.

Table 4. Quantitative results of unconditional generation. COV, IR, and Novel are reported in percentage (%).

Method	MMD ↓	JSD ↓	COV ↑	IR ↓	Novel ↑
DeepCAD	1.45	3.11	74.13	23.57	90.8
SkexGen	1.13	1.05	78.32	28.82	98.0
HNC-CAD	0.99	0.69	83.59	21.71	91.9
CAD-Factory	0.92	0.68	79.47	0.98	92.3

As shown in Table 4, our method significantly lowers the invalid rate and achieves superior MMD and JSD. Qualitative comparisons in Figure 7 further show that CAD-Factory generates structurally diverse and geometrically intricate CAD models, indicating strong potential for early-stage design exploration.

4.2.4 CAD Editing. We evaluate CAD-Factory on a range of CAD editing tasks, as shown in Figure 8 (left). Starting from an initial result, the model is refined through several editing instructions. Each instruction combines the current CAD program with localization cues derived from part-specific renderings, which map visual highlights to corresponding code components and enable structure-aware edits. In addition, we demonstrate the model’s capacity for diversity modeling in Figure 8 (right), where multiple valid CAD variants are generated from semantically similar descriptions. This capability facilitates design exploration by allowing users to select and iteratively refine preferred candidates into precise CAD models. See Appendix D.1 for more editing examples.

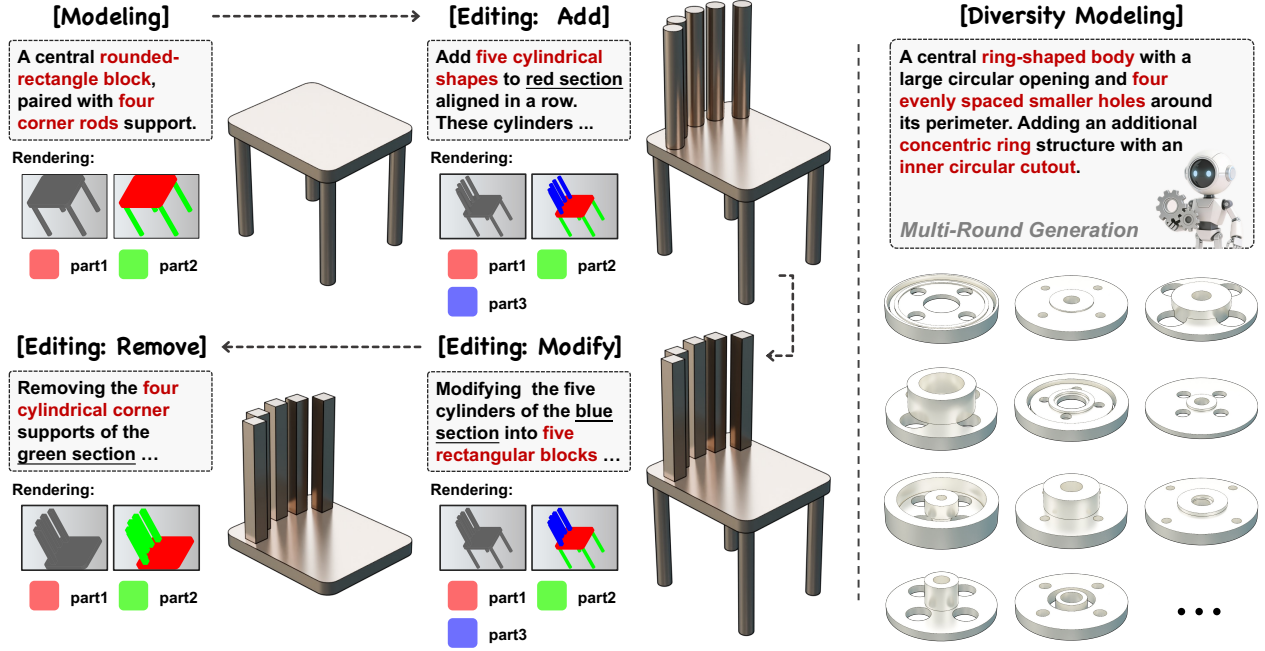


Fig. 8. CAD editing and design exploration, illustrating language-guided editing (left) and multi-round generation of diverse design alternatives (right).

Table 5. Ablation study for text-to-CAD generation.

Method	ACC _{ast} ↑	F1 ↑	MCD ↓	MMD ↓	JSD ↓
w/o Text-to-AST	79.60	72.69	6.37	1.53	1.91
w/o Text-to-AST*	80.16	73.26	3.35	1.18	1.21
w/o CAD DAPT	81.18	71.61	13.24	1.79	5.99
CAD-Factory	82.81	76.06	2.90	1.22	1.18

4.3 Ablation Study

To assess the contribution of key components in CAD-Factory, we conduct ablation studies under three settings: (1) removing the planning agent and directly performing text-to-code generation (w/o Text-to-AST); (2) replacing the coding agent with a stronger backbone (*LLaMA-3-8B* [Grattafiori et al. 2024]) while still omitting the planning stage (w/o Text-to-AST*); and (3) skipping CAD domain-adaptive pretraining (w/o CAD DAPT). As reported in Table 5, removing explicit AST generation substantially degrades structural accuracy, leading to notable drops in parameter correctness and geometric quality—an issue not remedied by simply increasing model capacity. These results suggest that the benefit of the AST is not merely structural regularization at the output level; rather, it acts as an intermediate scaffold that reduces the difficulty of jointly inferring program organization and geometric parameters from under-constrained language input. Moreover, omitting CAD-specific domain-adaptive pretraining introduces a clear distribution mismatch, resulting in reduced consistency and precision.

5 Conclusion

We present CAD-Factory, a language-driven CAD generation system that bridges high-level language descriptions and structured CAD code synthesis. By introducing an intermediate AST representation for staged generation, CAD-Factory separates program-level structural planning from parameter completion, thereby reducing reasoning complexity and enabling more interpretable, controllable, and editable CAD modeling. To support staged CAD synthesis, we further develop a process-oriented Text-CAD dataset with parameter-free/partial annotations. Extensive experiments demonstrate that CAD-Factory consistently outperforms strong baselines while naturally supporting a broad range of downstream CAD applications.

Limitations. The current work focuses on “*sketch-extrusion*” modeling. The proposed system is extensible and can be adapted to other CAD operations given appropriate data. A remaining challenge lies in the inherent ambiguity of natural language, which complicates the generation of parameter-accurate models.

Acknowledgments

The work of NTU researchers is supported by the MOE AcRF Tier 1 Grant of Singapore under Grant RG107/25. The work of BIT researchers is partially supported by the Beijing Natural Science Foundation under Grant L233003, the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China under Grant JYB2025XDXM206, and the China Scholarship Council under Grant 202406030027.

References

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmerschmidt, Sam Altman, Shyamal Anadkat, et al. 2024. GPT-4 Technical Report. arXiv:2303.08774
- Kamel Alrashedy, Pradyumna Tambwekar, Zulfiqar Haider Zaidi, Megan Langwasser, Wei Xu, and Matthew Gombolay. 2025. Generating CAD Code with Vision-Language Models for 3D Designs. In *The Thirteenth International Conference on Learning Representations (ICLR)*. Singapore, 52236–52262.
- Cheng Chen, Jiacheng Wei, Tianrun Chen, Chi Zhang, Xiaofeng Yang, Shangzhan Zhang, Bingchen Yang, Chuan-Sheng Foo, Guosheng Lin, Qixing Huang, and Fayao Liu. 2025b. CADCrafter: Generating Computer-Aided Design Models from Unconstrained Images. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Vancouver, BC, Canada, 11073–11082. doi:10.1109/CVPR52734.2025.01034
- Jingye Chen, Zhaoen Wang, Nanxuan Zhao, Li Zhang, Difan Liu, Jimei Yang, and Qifeng Chen. 2025a. Rethinking Layered Graphic Design Generation with a Top-Down Approach. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, Honolulu, HI, USA, 16861–16870.
- Tianrun Chen, Chunyan Yu, Yuanqi Hu, Jing Li, Tao Xu, Runlong Cao, Lanyun Zhu, Ying Zang, Yong Zhang, Zejian Li, and Lingyun Sun. 2025c. Img2CAD: Conditioned 3-D CAD Model Generation From Single Image With Structured Visual Geometry. *IEEE Trans. Ind. Inform.* 21, 11 (2025), 8539–8549.
- Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *The Twelfth International Conference on Learning Representations (ICLR)*, Vol. 2024. Vienna, Austria.
- Yuhao Du, Shunian Chen, Wenbo Zan, Peizhao Li, Mingxuan Wang, Dingjie Song, Bo Li, Yan Hu, and Benyou Wang. 2024. BlenderLLM: Training Large Language Models for Computer-Aided Design with Self-improvement. arXiv:2412.14203
- Elona Dupont, Kseniya Cherenkova, Dimitrios Mallis, Gleb Gusev, Anis Kacem, and Djamilia Aouada. 2024. Transcad: A hierarchical transformer for cad sequence inference from point clouds. In *Proceedings of the European conference on computer vision (ECCV)*. Springer Nature Switzerland, Milan, Italy, 19–36. doi:10.1007/978-3-031-73030-6_2
- Mohamed Elrefaie, Janet Qian, Raina Wu, Qian Chen, Angela Dai, and Faez Ahmed. 2025. AI agents in engineering design: a multi-agent framework for aesthetic and aerodynamic car design. In *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, Vol. 89237. Los Angeles CA, USA, V03BT03A048.
- Fengxiao Fan, Jingzhe Ni, Xiaolong Yin, Sirui Wang, Xingyu Lu, Qiang Zou, Ruofeng Tong, Min Tang, and Peng Du. 2025. CADDesigner: Conceptual Design of CAD Models Based on General-Purpose Agent. arXiv:2508.01031 [cs.AI] <https://arxiv.org/abs/2508.01031>
- Rao Fu, Qian Li, Cheng Wen, Ning An, and Fulin Tang. 2025. A Novel Framework for Learning Bézier Decomposition From 3D Point Clouds. *IEEE Trans. Circuits Syst. Video Technol.* 35, 5 (2025), 4329–4340.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783
- Mikell P. Groover and Emory W. Zimmers. 1984. *CAD/CAM: Computer-Aided Design and Manufacturing*. Prentice-Hall, Englewood Cliffs, NJ.
- Yandong Guan, Xilin Wang, XiMing Xing, Jing Zhang, Dong Xu, and Qian Yu. 2025. CAD-Coder: Text-to-CAD Generation with Chain-of-Thought and Geometric Reward. In *Advances in Neural Information Processing Systems (NeurIPS)*. Curran Associates, Inc., San Diego, CA, USA.
- Haoxiang Guo, Shilin Liu, Hao Pan, Yang Liu, Xin Tong, and Baining Guo. 2022. ComplexGen: CAD reconstruction by B-rep chain complex generation. *ACM Trans. Graph.* 41, 4, Article 129 (2022), 18 pages.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. 2024. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations (ICLR)*, Vol. 2024. Vienna, Austria.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models.. In *The Tenth International Conference on Learning Representations (ICLR)*. Virtual.
- Di Huang, Ziyuan Nan, Xing Hu, Pengwei Jin, Shaohui Peng, Yuanbo Wen, Rui Zhang, Zidong Du, Qi Guo, Yewen Pu, and Yunji Chen. 2023. ANPL: Towards Natural Programming with Interactive Decomposition. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36. Curran Associates, Inc., New Orleans, LA, USA, 69404–69440.
- Benjamin T Jones, Zihan Zhang, Felix Hähnlein, Wojciech Matusik, Maaz Ahmad, Vladimir Kim, and Adriana Schulz. 2025. A Solver-Aided Hierarchical Language for LLM-Driven CAD Design. In *Computer Graphics Forum*, Vol. 44. Wiley Online Library, e70250.
- Mohammad Sadil Khan, Elona Dupont, Sk Aziz Ali, Kseniya Cherenkova, Anis Kacem, and Djamilia Aouada. 2024a. CAD-SIGNet: CAD Language Inference from Point Clouds Using Layer-Wise Sketch Instance Guided Attention. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Seattle, WA, USA, 4713–4722. doi:10.1109/CVPR52733.2024.00451
- Mohammad Sadil Khan, Sankalp Sinha, Talha Uddin Sheikh, Didier Stricker, Sk Aziz Ali, and Muhammad Zeshan Afzal. 2024b. Text2CAD: Generating Sequential CAD Designs from Beginner-to-Expert Level Text Prompts. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 37. Curran Associates, Inc., Vancouver, BC, Canada, 7552–7579. doi:10.52202/079017-0242
- Joseph G. Lambourne, Karl D. D. Willis, Pradeep Kumar Jayaraman, Aditya Sanghi, Peter Meltzer, and Hooman Shayani. 2021. BRepNet: A topological message passing system for solid models. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Nashville, TN, USA, 12768–12777.
- Bingxuan Li, Yiwei Wang, Jiuxiang Gu, Kai-Wei Chang, and Nanyun Peng. 2025b. METAL: A Multi-Agent Framework for Chart Generation with Test-Time Scaling. In *ACL (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 30054–30069.
- Changjian Li, Hao Pan, Adrien Bousseau, and Niloy J. Mitra. 2022. Free2CAD: parsing freehand drawings into CAD commands. *ACM Trans. Graph.* 41, Article 93 (2022), 16 pages. doi:10.1145/3528223.3530133
- Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, Honolulu, HI, USA, 19730–19742.
- Jiahao Li, Weijian Ma, Xueyang Li, Yunzhong Lou, Guichun Zhou, and Xiangdong Zhou. 2025a. CAD-Llama: Leveraging Large Language Models for Computer-Aided Design Parametric 3D Model Generation. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Vancouver, BC, Canada, 18563–18573. doi:10.1109/CVPR52734.2025.01730
- Pu Li, Wenhao Zhang, Jianwei Guo, Jinglu Chen, and Dong-Ming Yan. 2025c. Revisiting CAD Model Generation by Learning Raster Sketch. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, Vol. 39. AAAI Press, Philadelphia, PA, USA, 4869–4877. doi:10.1609/aaai.v39i5.32515
- Pu Li, Wenhao Zhang, Weize Quan, Biao Zhang, Peter Wonka, and Dong-Ming Yan. 2025d. BrepGPT: Autoregressive B-rep Generation with Voronoi Half-Patch. *ACM Trans. Graph.* 44, 6 (2025), 226:1–226:18.
- Xueyang Li, Yu Song, Yunzhong Lou, and Xiangdong Zhou. 2024. CAD Translator: An Effective Drive for Text to 3D Parametric Computer-Aided Design Generative Modeling. In *Proceedings of the 32nd ACM International Conference on Multimedia (MM)*. Association for Computing Machinery, Melbourne VIC, Australia, 8461–8470.
- Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Cheng-gang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437
- Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *The Seventh International Conference on Learning Representations (ICLR)*. New Orleans, LA, USA.
- Dimitrios Mallis, Ahmet Serda Karadeniz, Sebastian Cavada, Danila Rukhovich, Niki Foteinopoulou, Kseniya Cherenkova, Anis Kacem, and Djamilia Aouada. 2025. CAD-Assistant: Tool-Augmented VLLMs as Generic CAD Task Solvers. In *2025 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, Honolulu, HI, USA, 7284–7294.
- AI Meta. 2024. Llama 3.2: Revolutionizing edge ai and vision with open, customizable models. *Meta AI Blog*. Retrieved December 20 (2024), 2024.
- Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. In *The Eleventh International Conference on Learning Representations (ICLR)*. Kigali, Rwanda.
- Felix Ocker, Stefan Menzel, Ahmed Sadik, and Thiago Rios. 2025. From Idea to CAD: A Language Model-Driven Multi-Agent System for Collaborative Design. arXiv:2503.04417
- Wamiq Para, Shariq Bhat, Paul Guerrero, Tom Kelly, Niloy Mitra, Leonidas J Guibas, and Peter Wonka. 2021. SketchGen: Generating Constrained CAD Sketches. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34. Curran Associates, Inc., Red Hook, NY, USA, 5077–5088.
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, Vol. 139. PMLR, Brookline, MA, USA, 8748–8763.
- Daxuan Ren, Jianmin Zheng, Jianfei Cai, Jiatong Li, Haiyong Jiang, Zhongang Cai, Junzhe Zhang, Liang Pan, Mingyuan Zhang, Haiyu Zhao, and Shuai Yi. 2021. CSG-Stump: A Learning Friendly CSG-Like Representation for Interpretable Shape Parsing. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, Montreal, QC, Canada, 12458–12467. doi:10.1109/ICCV48922.2021.01225

- Daxuan Ren, Jianmin Zheng, Jianfei Cai, Jiatong Li, and Junzhe Zhang. 2022. Extrudenet: Unsupervised inverse sketch-and-extrude for shape parsing. In *Proceedings of the European conference on computer vision (ECCV)*. Springer Nature Switzerland, Tel Aviv, Israel, 482–498. doi:10.1007/978-3-031-20086-1_28
- Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2024. Code Llama: Open Foundation Models for Code. arXiv:2308.12950
- Danila Rukhovich, Elona Dupont, Dimitrios Mallis, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. 2025. CAD-Recode: Reverse Engineering CAD Code from Point Clouds. In *2025 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, Honolulu, HI, USA, 9801–9811.
- Jami J. Shah and Martti Mäntylä. 1995. *Parametric and Feature-Based CAD/CAM: Concepts, Techniques, and Applications*. John Wiley & Sons, Chichester, UK.
- Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv:2403.05530
- Ruiyu Wang, Shizhao Sun, Weijian Ma, and Jiang Bian. 2026. CAD-Tokenizer: Towards Text-based CAD Prototyping via Modality-Specific Tokenization. arXiv:2509.21150 [cs.LG] <https://arxiv.org/abs/2509.21150>
- Ruiyu Wang, Yu Yuan, Shizhao Sun, and Jiang Bian. 2025b. Text-to-CAD Generation Through Infusing Visual Feedback in Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, Vol. 267. PMLR, Vancouver, BC, Canada, 65326–65345.
- Siyu Wang, Cailian Chen, Xinyi Le, Qimin Xu, Lei Xu, Yanzhou Zhang, and Jie Yang. 2025a. CAD-GPT: Synthesising CAD Construction Sequence with Spatial Reasoning-Enhanced Multimodal LLMs. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, Vol. 39. AAAI Press, Philadelphia, PA, USA, 7880–7888. doi:10.1609/aaai.v39i8.32849
- Xilin Wang, Jia Zheng, Yuanchao Hu, Hao Zhu, Qian Yu, and Zihan Zhou. 2025c. From 2d cad drawings to 3d parametric models: A vision-language approach. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, Vol. 39. AAAI Press, Philadelphia, PA, USA, 7961–7969. doi:10.1609/aaai.v39i8.32858
- Karl D. D. Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. 2021. Fusion 360 gallery: a dataset and environment for programmatic CAD construction from human design sequences. *ACM Trans. Graph.* 40, 4, Article 54 (2021), 24 pages.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In *First Conference on Language Modeling*. Philadelphia, PA, USA.
- Rundi Wu, Chang Xiao, and Changxi Zheng. 2021. DeepCAD: A Deep Generative Network for Computer-Aided Design Models. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, Montreal, QC, Canada, 6752–6762. doi:10.1109/ICCV48922.2021.00670
- Jingwei Xu, Chenyu Wang, Zibo Zhao, Wen Liu, Yi Ma, and Shenghua Gao. 2025. CAD-MLLM: Unifying Multimodality-Conditioned CAD Generation With MLLM. arXiv:2411.04954
- Xiang Xu, Pradeep Kumar Jayaraman, Joseph George Lambourne, Karl D.D. Willis, and Yasutaka Furukawa. 2023. Hierarchical Neural Coding for Controllable CAD Model Generation. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, Vol. 202. PMLR, Honolulu, HI, USA, 38443–38461.
- Xiang Xu, Joseph Lambourne, Pradeep Jayaraman, Zhengqing Wang, Karl Willis, and Yasutaka Furukawa. 2024. BrepGen: A B-rep Generative Diffusion Model with Structured Latent Geometry. *ACM Trans. Graph.* 43, Article 119 (2024), 14 pages.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025b. Qwen3 Technical Report. arXiv:2505.09388
- Bingchen Yang, Haiyong Jiang, Hao Pan, Guosheng Lin, Jun Xiao, and Peter Wonka. 2025a. PS-CAD: Local Geometry Guidance via Prompting and Selection for CAD Reconstruction. *ACM Trans. Graph.* 44, 3 (2025), 21 pages.
- Mohsen Yavartanoo, Sangmin Hong, Reyhaneh Neshatavar, and Kyoung Mu Lee. 2024. Text2CAD: Text to 3D CAD Generation via Technical Drawings. arXiv:2411.06206
- Zeqing Yuan, Haoxuan Lan, Qiang Zou, and Junbo Zhao. 2024. 3D-PreMise: Can Large Language Models Generate 3D Shapes with Sharp Features and Parametric Control? arXiv:2401.06437
- Zhanwei Zhang, Shizhao Sun, Wenxiao Wang, Deng Cai, and Jiang Bian. 2025. FlexCAD: Unified and Versatile Controllable CAD Generation with Fine-tuned Large Language Models. In *The Thirteenth International Conference on Learning Representations (ICLR)*. Singapore, 3204–3227.